

CSA15 · CLOUD COMPUTING & BIG DATA ANALYTICS

Cloud Computing & Virtualization

Analytical Problem Solving — Independent Edition

Topics covered: hypervisor architecture · tenant isolation · performance bottleneck diagnosis · software-defined datacenters · multi-cloud identity federation

Name: Yarra Pawan
Roll No / ID: 192424294
Class / Section: CSA1503
Date: 03-08-2026 _____

How to Use This Workbook

This is an independent, self-authored edition of the CSA15 CO2 Analytical Problem-Solving assessment. It answers the same five set questions with a fresh line of reasoning, different supporting examples, and its own diagrams, so it should be treated as an original piece of work rather than a copy of any other submission.

Each task below contains:

- The original assessment question, quoted for reference in a teal PROMPT callout.
- An independent analytical discussion — architecture, comparison tables, and reasoning written from scratch.
- A generated concept diagram ("Fig.") that is already embedded in the document.
- An orange dashed "EVIDENCE NEEDED" box telling you exactly which tool output, dashboard, or command screenshot to capture and insert before submission.

To insert a screenshot: click inside the dashed box, delete the placeholder text, then use Insert > Pictures in Word and resize the image to fit within the box borders.

Compare Type-1 and Type-2 hypervisors for a cloud datacenter hosting mission-critical application. Analyze performance, security, and manageability, then justify the better choice.

1.1 Two Different Places to Put the Hypervisor

A Type-1 (bare-metal) hypervisor — for example VMware ESXi, Microsoft Hyper-V, Citrix Hypervisor, or KVM — installs directly on the physical server and owns the CPU, memory, storage and network hardware itself. A Type-2 (hosted) hypervisor — for example Oracle VirtualBox, Parallels Desktop, or QEMU running under a desktop OS — is just another application that a full-blown host operating system (Windows/Linux/macOS) has to schedule, so every virtual machine's requests are filtered through that host OS first.

1.2 Side-by-Side Comparison

Dimension	Type-1 (Bare-Metal)	Type-2 (Hosted)
Performance overhead	Very low — VM instructions reach hardware through a thin control layer using Intel VT-x/AMD-V.	Higher and less predictable — every request queues behind the host OS scheduler and its background processes.
Attack surface	Small, purpose-built codebase; no general-purpose OS services to exploit.	Inherits every vulnerability of the underlying desktop/server OS plus its drivers and apps.
Scale-out management	Native clustering, live migration and automated failover (vCenter/SCVMM/oVirt-style tooling).	Built for a single machine; no built-in clustering or centralized policy engine.
Recovery from a crash	A hypervisor fault is contained to that host; cluster HA restarts affected VMs elsewhere.	A host-OS crash or forced reboot (e.g., patch Tuesday) takes every guest VM down with it.
Best fit	Production IaaS, banking/health/government workloads, anything with an SLA.	Developer laptops, quick demos, isolated training labs.

1.3 Verdict

For a datacentre that must guarantee uptime and defend regulated data, a Type-1 hypervisor is the only defensible choice. Its shorter I/O path protects the latency budgets that mission-critical SLAs depend on, its minimal codebase shrinks the attack surface a compromise would need to exploit, and its clustering features (live migration, automated failover) directly satisfy the availability requirements that banking, healthcare, or government workloads are held to. Type-2 hypervisors stay useful for sandboxing and training, but they should never sit under production traffic.

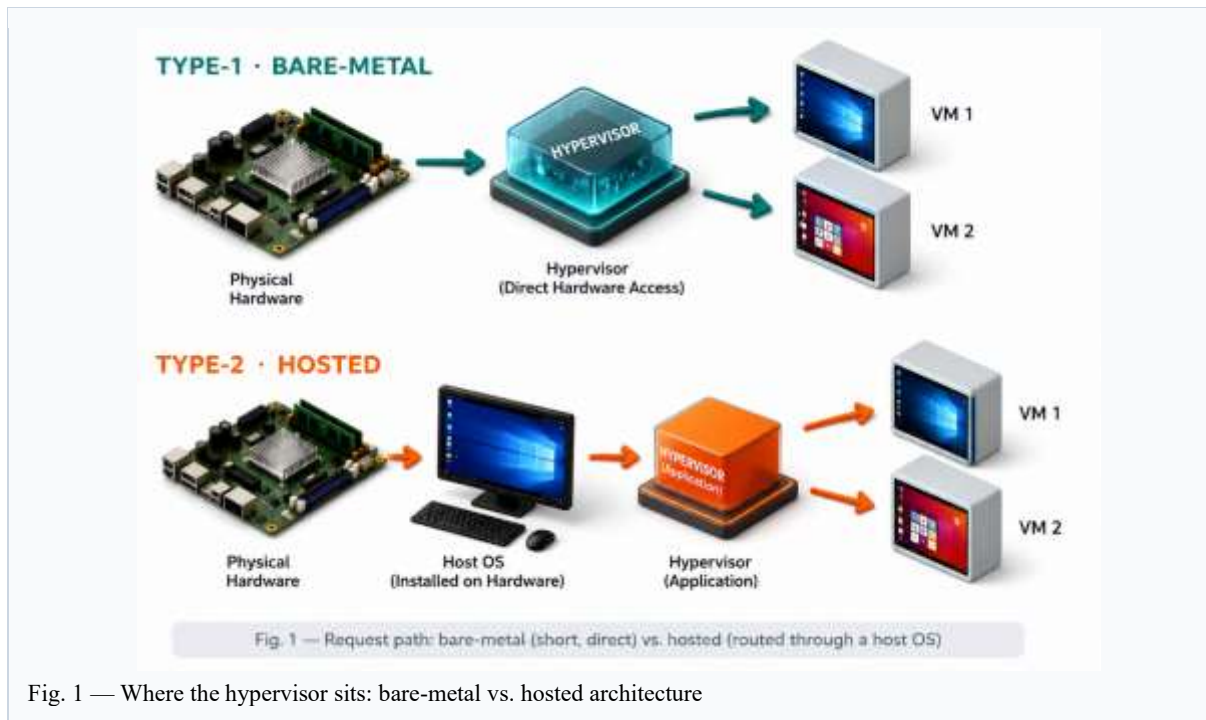
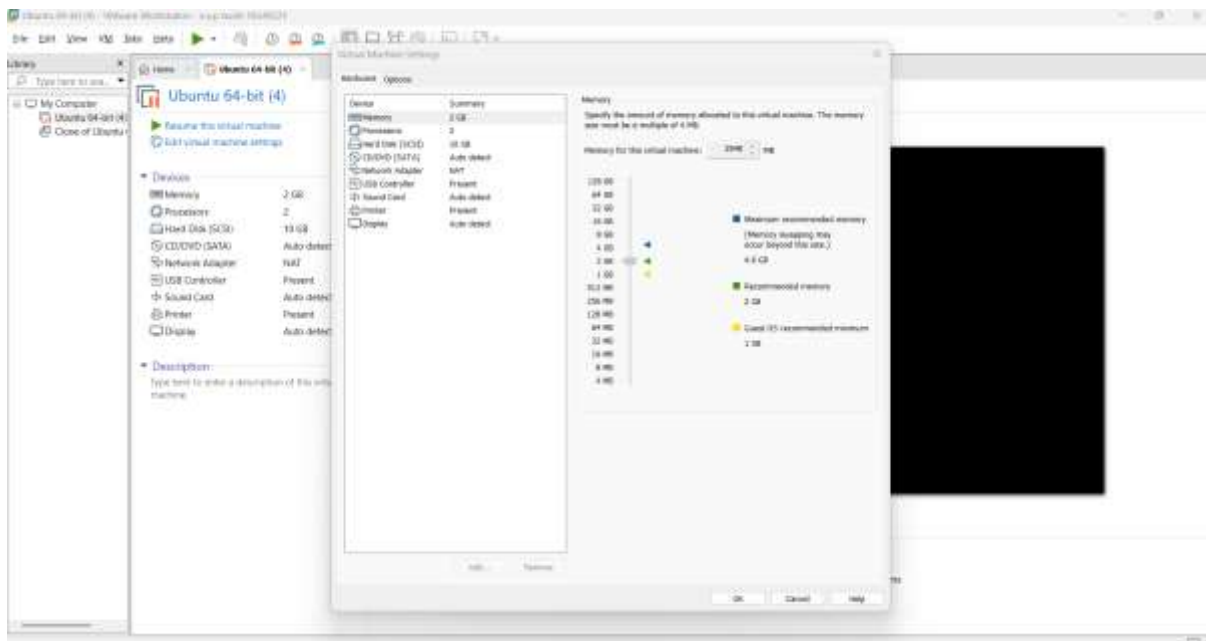
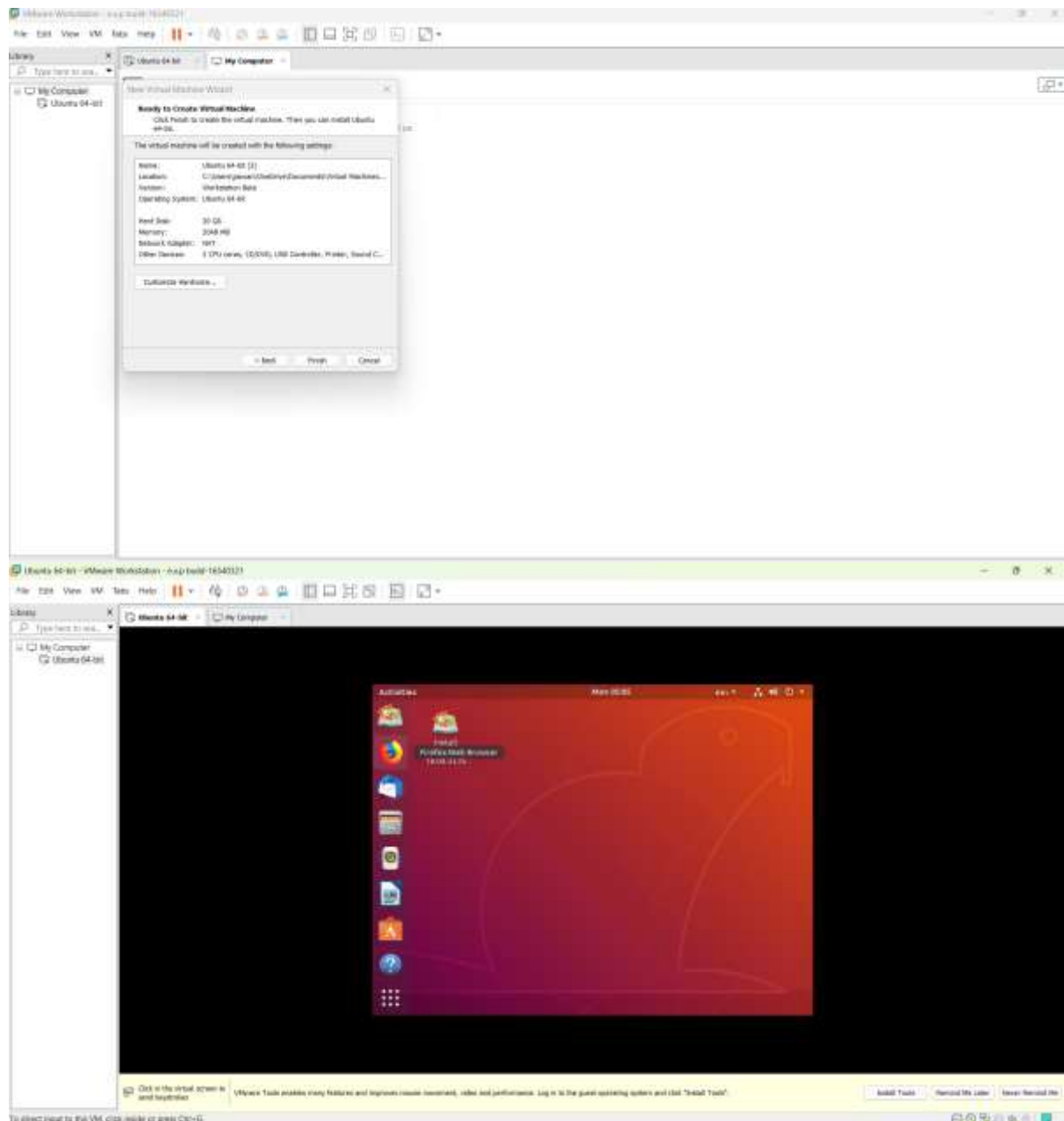


Fig. 1 — Where the hypervisor sits: bare-metal vs. hosted architecture





Task-2: Keeping Finance, Healthcare and Student Workloads Apart on Shared Hardware

2.1 Isolation Mechanisms, Layer by Layer

- **Compute isolation:** each tenant gets its own virtual CPU and RAM allocation; the hypervisor's scheduler enforces reservations/limits/shares so a spike in the student portal cannot starve the healthcare VM of cycles.
- **Memory/DMA isolation:** Extended Page Tables and an IOMMU (VT-d/AMD-Vi) stop one VM from reading another VM's physical memory even through a directly-attached device.
- **Network isolation:** per-tenant VLANs/VXLANs plus distributed firewall rules at the virtual switch keep finance, healthcare and student traffic in separate broadcast domains, with micro-segmentation blocking east-west traffic between them.
- **Storage isolation:** separate virtual disks and per-tenant encryption keys mean that even a shared SAN/NAS cannot be read across tenants without the right key.
- **Compliance segregation:** HIPAA (healthcare) and PCI-DSS (finance) typically push audit logs, key management and access control into separate management domains or dedicated clusters rather than relying on VM boundaries alone.

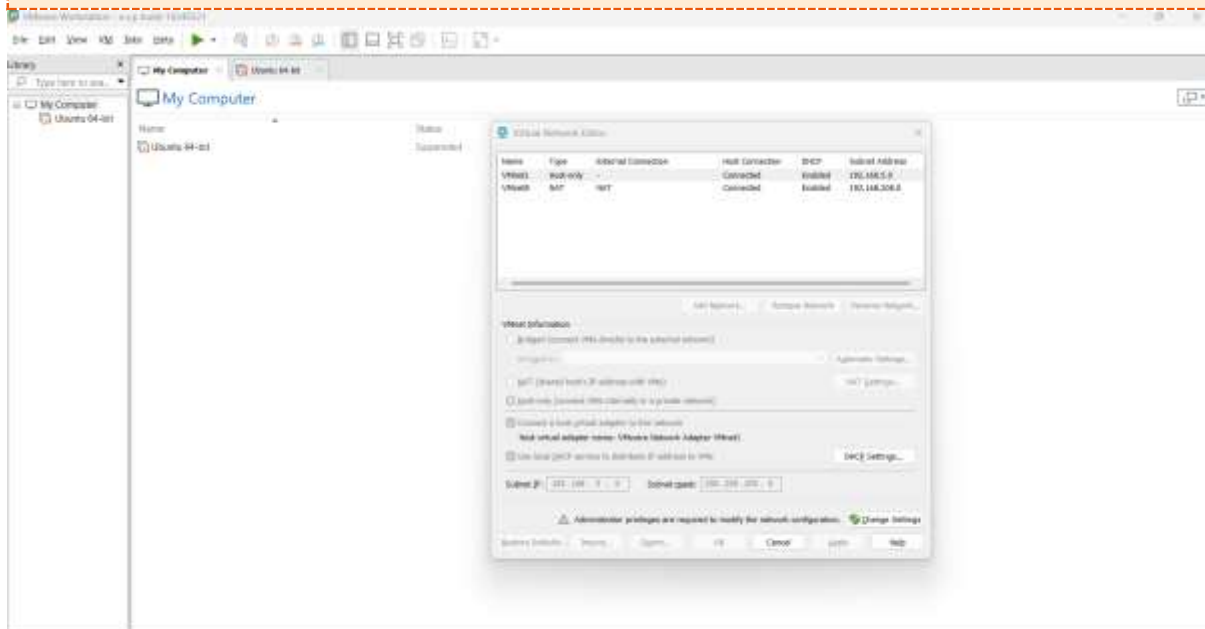
2.2 Two Attack Surfaces Worth Flagging

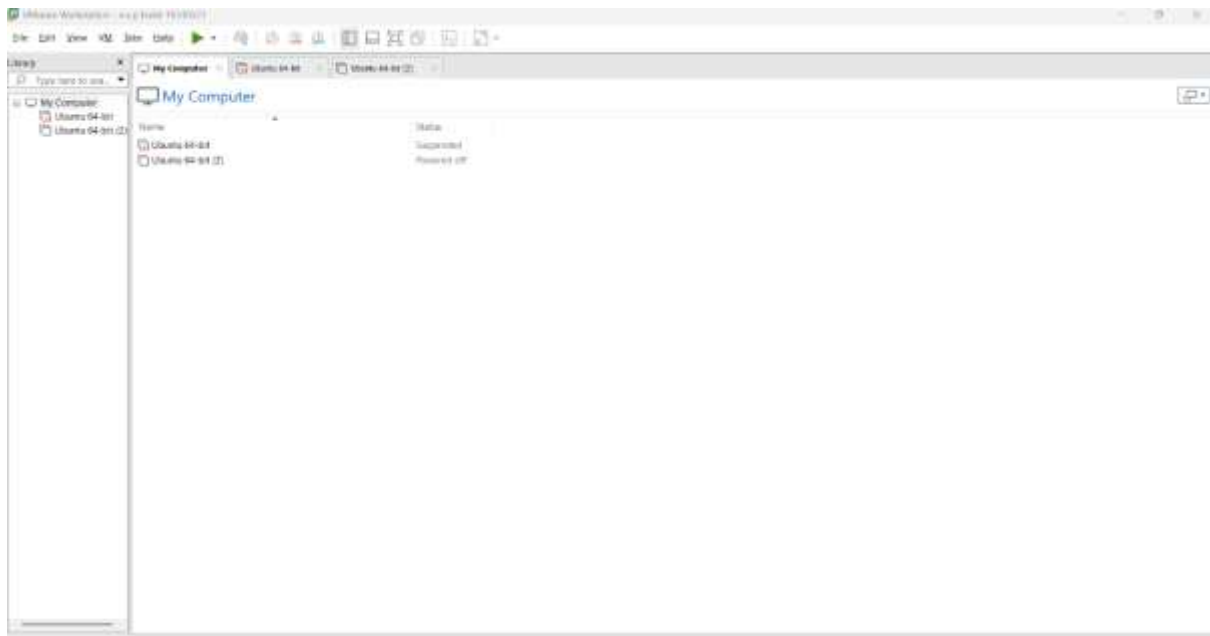
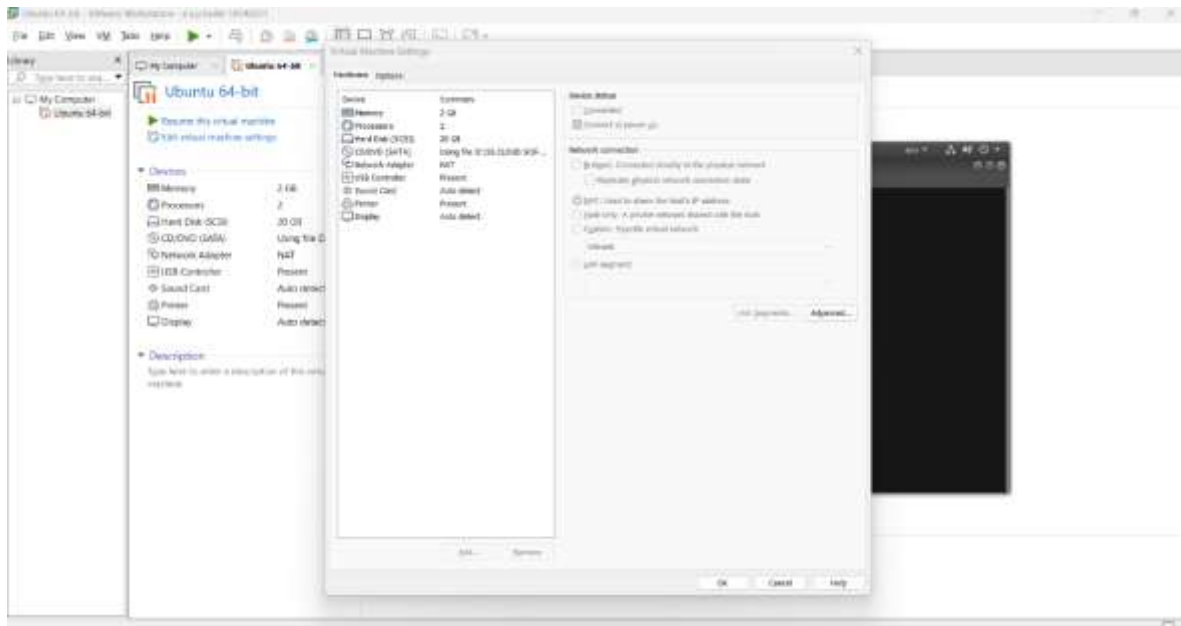
Attack Surface	Why It Matters Here
Cross-VM side-channel leakage (cache-timing / Spectre-class attacks)	Co-resident VMs share physical CPU caches and branch predictors. A malicious student-portal VM placed on the same core as the finance VM could, in principle, infer secrets through timing differences — without ever breaking the hypervisor's formal isolation boundary.
Compromised orchestration / cloud control-plane API	The API that provisions, snapshots and migrates VMs sits outside any single tenant's boundary. Stolen API credentials or a flaw in that control plane could let an attacker clone or relocate a healthcare VM regardless of how well the hypervisor itself isolates memory and storage.

A secondary risk worth monitoring is misconfigured micro-segmentation — a single overly-broad firewall rule between VLANs can quietly reopen lateral movement between the three tenant classes.



Fig. 2 — Tenant VMs isolated by the hypervisor, with the two flagged attack surfaces





Task-3: Study the following VM host utilization snapshot and recommend corrective actions: CPU 92%, memory 88%, storage I/O latency 35 ms, average VM boot time 170 s. Explain the likely bottlenecks.

3.1 Snapshot vs. Healthy Range

Metric	Observed	Healthy Range	Verdict
CPU utilization	92%	< 70–80% sustained	Over budget
Memory utilization	88%	< 80–85% sustained	Over budget
Storage I/O latency	35 ms	< 10–20 ms (SSD/SAN)	2–3x too slow
Average VM boot time	170 s	30–60 s typical	3–5x too slow

3.2 Reading the Chain of Causation

These four numbers are not four independent problems — they describe one overloaded host. Read them as a chain:

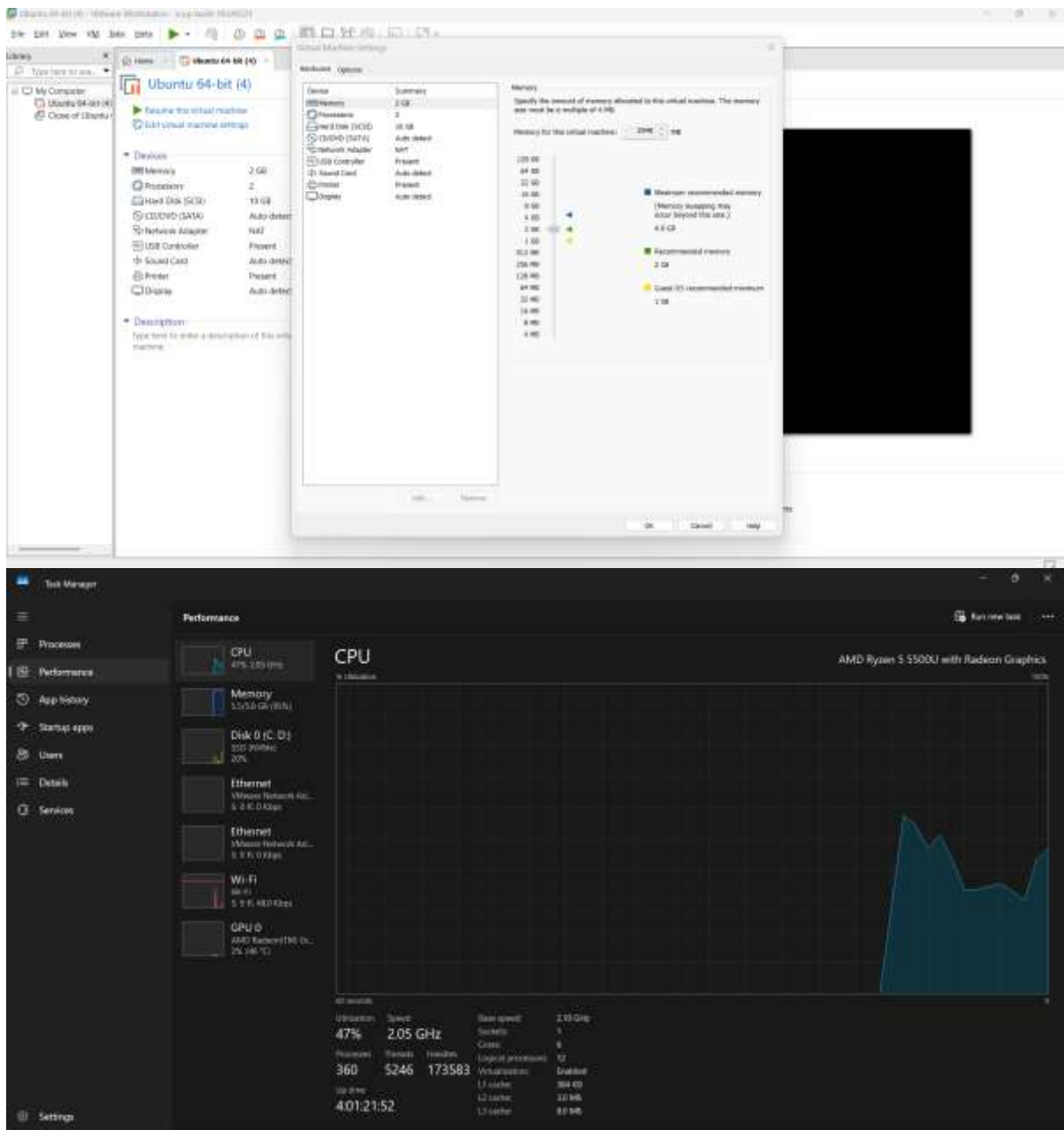
- CPU sitting at 92% means VMs are queuing for physical cores ("CPU ready" time), which alone slows every operation, boot included.
- Memory at 88% forces the hypervisor into reclamation — ballooning, compression, and in the worst case swapping guest memory to disk, which turns RAM-speed operations into disk-speed operations.
- 35 ms storage latency (2–3x the healthy ceiling) is consistent with that swap traffic landing on a datastore that is already close to its IOPS limit, or too many VMs sharing one datastore.
- 170 s boot time is the compound symptom: a booting VM needs CPU cycles (contended), memory (contended, possibly triggering swap) and a burst of sequential disk reads (already latent) all at once.

In short: the host is over-committed. Memory pressure pushes work onto an already-saturated storage layer, which burns extra CPU cycles on swap/balloon management, which worsens the CPU contention — and boot time absorbs all of it.

3.3 Corrective Actions, by Urgency

- **Right now:** trigger DRS-style automated load balancing to move VMs off this host, and identify/throttle the "noisy neighbor" VM(s) via resource pools, shares and limits.
- **This week:** right-size VM memory allocations to cut reliance on ballooning, and add physical RAM or lower the memory over-commitment ratio.
- **This week:** move the busiest VMs to an NVMe/SSD tier or raise provisioned IOPS; check queue depth and multipathing on the storage path.
- **This quarter:** do capacity planning — add hosts/cores to the cluster once aggregate demand consistently exceeds ~75–80%.
- **This quarter:** switch frequently-rebuilt templates to instant-clone/linked-clone provisioning to cut disk reads during boot storms.
- **Ongoing:** alert on CPU-ready time, balloon/swap activity and storage latency so this pattern is caught before it hits boot times again.





Task-4: Analyze how Software Defined Datacenter architecture improves agility compared with a traditional datacenter. Support your answer with compute, storage, and network virtualization considerations.

4.1 The Core Shift

A traditional datacenter ties every capability to a specific physical box: a new application means racking a server, wiring a SAN LUN, and hand-configuring a switch port — each step owned by a different specialist team. A Software Defined Datacenter (SDDC) pulls compute, storage and networking into software-managed pools that sit behind one policy-driven, API-accessible control plane (tools such as OpenStack, Nutanix, or VMware Cloud Foundation are common implementations), so provisioning becomes a request to that API instead of a ticket to three different teams.

4.2 Agility, Layer by Layer

Layer	Traditional Datacenter	SDDC
Compute	New capacity = purchase, rack and cable a server (days–weeks).	VMs/containers spun up from templates via self-service portal or API in minutes; live-migrated without downtime.
Storage	Capacity/performance changes need vendor-specific array tools.	Pooled storage (e.g., vSAN/Ceph) with policy tiers (gold/silver/bronze) applied instantly in software.
Network	VLANs and firewall rules hand-configured on physical switches — usually the slowest step.	Virtual networks, micro-segmentation and load balancers (SDN/NSX-style) created or torn down in minutes.

4.3 What This Buys the Organization

- **Provisioning speed:** a full compute+storage+network environment deploys from an infrastructure-as-code template in minutes rather than a days-to-weeks change cycle.
- **Elastic cost:** pools scale up or down with demand instead of being permanently sized for a hardware purchase.
- **Consistent policy:** one set of security/compliance/QoS rules is defined once and enforced everywhere, instead of being reconciled across separate hardware consoles.
- **Vendor flexibility:** hardware can be refreshed or mixed across vendors underneath the same software layer without disrupting running services.
- **Faster recovery:** because the whole environment is defined as code, it can be redeployed to a DR site far faster than rebuilding physical configuration by hand.

Conclusion: an SDDC does not add new hardware capability so much as it removes the manual, siloed steps between wanting a resource and having it — and that removal is exactly what "agility" means in this context.

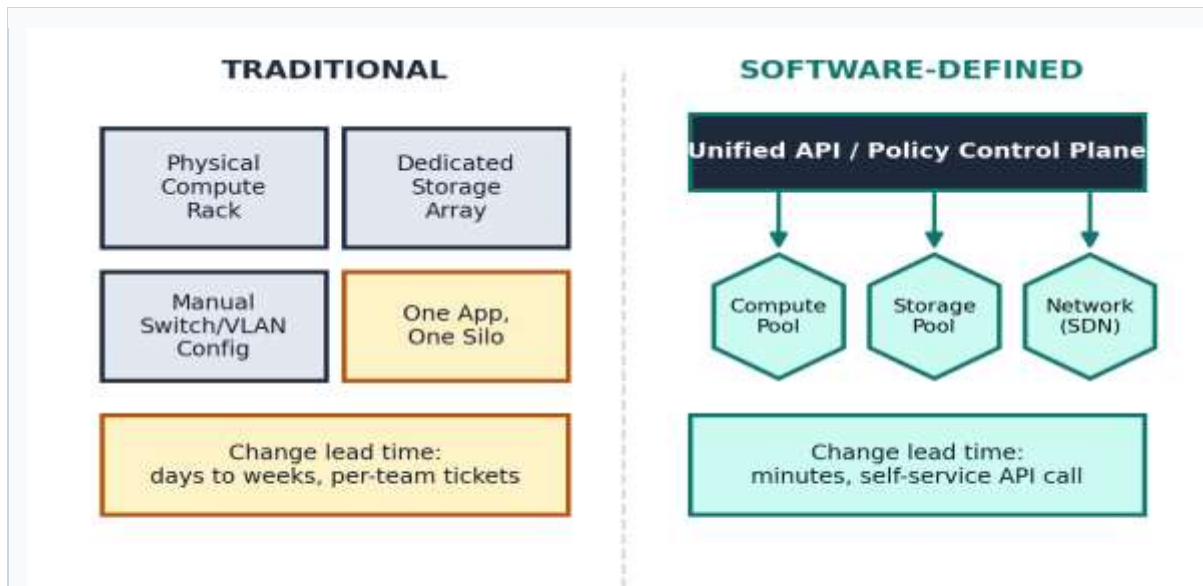


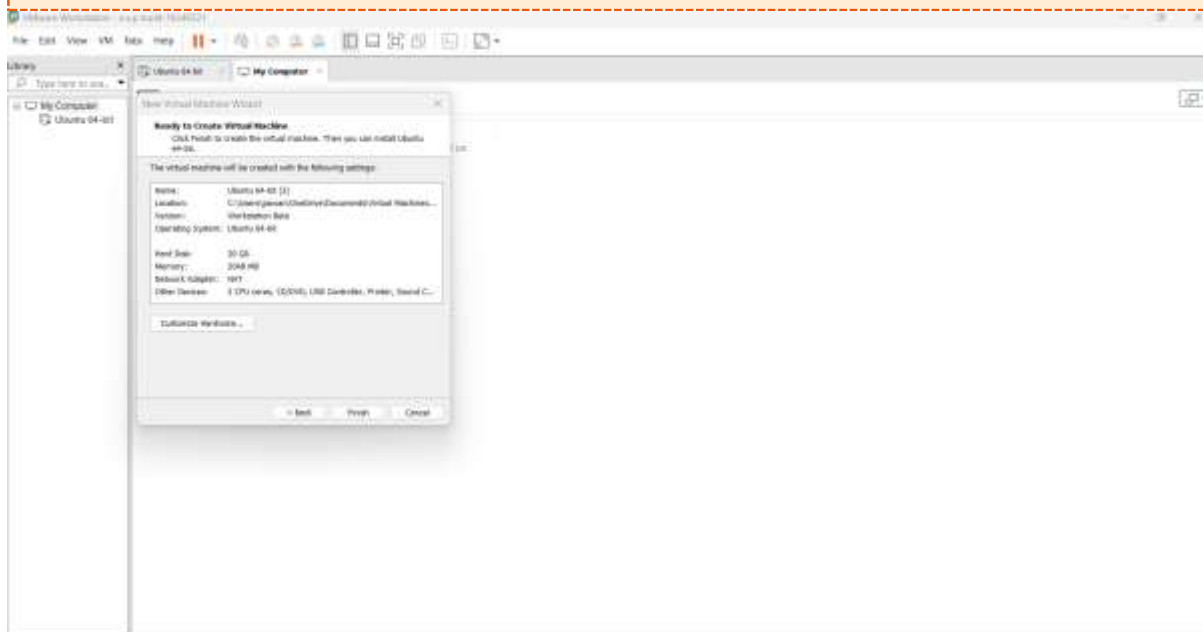
Fig. 4 — Siloed hardware provisioning (left) vs. pooled, API-driven SDDC (right)

Fig. 4 — Siloed provisioning vs. pooled, API-driven SDDC provisioning

EVIDENCE NEEDED — Self-service provisioning evidence

Capture a screenshot of a template/blueprint-based VM deployment (e.g., a cloud console "launch instance" wizard, an OpenStack Horizon dashboard, or a Terraform apply output) showing a resource being provisioned in software rather than through manual hardware setup.

→ Insert > Pictures, then delete this placeholder text.



TASK-5: A multi-cloud federation project fails due to identity mismatches between providers. Analyze the issue and propose a secure identity and privacy design using federation concepts.

5.1 Why the Mismatch Happens

- **Different identity stores per cloud:** AWS IAM, Azure AD and a GCP/on-prem LDAP directory each name and structure user attributes differently (e.g., "department" vs. "org_unit"), so authorization rules that work on one provider silently fail on another.
- **No shared trust framework:** without a common standard (SAML 2.0 or OpenID Connect/OAuth 2.0) providers cannot verify each other's identity assertions, so users end up re-authenticating separately on every cloud.
- **Incompatible tokens:** differing token lifetimes, claim structures and signing algorithms mean an assertion issued by one provider's IdP can be rejected or misread by another's service provider.
- **No shared privacy/residency policy:** without a governance layer, attributes can cross borders or providers in ways that breach GDPR or similar rules, especially when release policies were never harmonized.

5.2 A Federation Design That Avoids This

- **Central identity broker:** put a broker (Keycloak, Okta, Ping Identity, or Azure AD B2B/B2C) between each provider's native IdP and the federation, translating claims into one common schema via SCIM.
- **Hub-and-spoke trust, not full mesh:** every cloud trusts signed assertions from the central broker instead of trusting every other cloud directly, so onboarding a new provider means one new trust relationship, not N new ones.
- **Canonical attribute schema:** a shared attribute dictionary (SCIM's standard user schema) ensures "role" or "clearance level" means the same thing regardless of which cloud issued the identity.
- **Short-lived signed tokens:** JWTs with rotation and mutually trusted certificate authorities let each provider verify an assertion cryptographically without a live call back to the issuing IdP.
- **Minimal disclosure (ABAC):** attribute-based access control releases only the attributes a given transaction actually needs, limiting cross-provider exposure of personal data.
- **MFA at the broker, once:** strong authentication happens centrally, so trust propagated outward is already high-assurance instead of being redundantly (and inconsistently) re-checked per cloud.
- **Central audit trail:** logging every federated login and attribute release at the broker supports GDPR/HIPAA/PCI-DSS reporting and speeds up incident investigation.

5.3 What This Fixes

- Single sign-on across every federated cloud, cutting credential sprawl and phishing exposure.
- Authorization decisions that behave the same regardless of which cloud is hosting the workload.
- Lower privacy risk, since only the minimum attribute set is ever released.
- Simple onboarding for new providers — they trust the broker, not every existing member individually.

